

Markdown Reference

For people who already know HTML

Markdown is just a shorthand for HTML. Every piece of Markdown compiles down to an HTML tag. If you know HTML, you already understand what every element *does* — you just need the new syntax. Where Markdown can't do something, you can drop raw HTML directly into any `.md` file and it works.

The Core Syntax

Headings

```
# H1      → <h1>
## H2     → <h2>
### H3    → <h3>
#### H4   → <h4>
##### H5  → <h5>
##### H6  → <h6>
```

Space after the `#` is required. GitHub READMEs rarely go below H3.

Text Formatting

```
**bold**      → <strong>
*italic*       → <em>
***bold italic***
~~strikethrough~~ → <del>   (GitHub Flavored Markdown only)
`inline code`    → <code>
```

Links and Images

```
[link text](https://url.com)          → <a href="...">
[link text](https://url.com "tooltip") → <a href="..." title="...">
![alt text](image.png)                → 
![alt text](image.png "tooltip")      → 
```

Reference-style links (cleaner for long docs):

```
[link text][ref-name]
```

```
[ref-name]: https://url.com
```

Lists

```
- item          → <ul><li>
- item
  - nested      (indent 2 spaces)

1. item         → <ol><li>
2. item
3. item
```

Mixing ordered and unordered lists works fine, and nesting them is just indentation.

Blockquotes

```
> This is a blockquote.      → <blockquote>
> It can span multiple lines.
>
> And multiple paragraphs.

> Nested:
>> inner quote
```

Code Blocks

Fenced with triple backticks. The language name after the opening fence enables **syntax highlighting** on GitHub:

```
```python
def hello():
 print("Hello, world!")
```

```bash
echo "hello"
```

```c
int main() { return 0; }
```
```

GitHub supports: `python`, `bash`, `c`, `cpp`, `javascript`, `typescript`, `html`, `css`, `json`, `yaml`, `toml`, `rust`, `go`, `java`, `sql`, `latex`, `diff`, and many more.

Horizontal Rule

```
--- → <hr>
```

Three or more hyphens, asterisks, or underscores. `---` is the convention.

Tables (GitHub Flavored Markdown)

Basic syntax:

```
Column 1	Column 2	Column 3
cell	cell	cell
cell	cell	cell
```

The separator row (`|---|`) is required — it's what tells the renderer this is a table. The number of dashes doesn't matter, minimum is one.

Column alignment — controlled by colons in the separator row:

```
Left	Center	Right
text	text	text
```

- `:---` = left (default if no colon)
- `:---:` = center
- `---:` = right

About borders on GitHub: Standard Markdown tables on GitHub *already render with visible borders* — GitHub's CSS adds them automatically. You don't need to do anything special for a normal bordered table. This is renderer-dependent though; the same Markdown in a different renderer (like a plain text editor preview) might show no borders at all.

Inline formatting works inside cells:

| | | |
|-----------------|------------------------------|--|
| Feature | Support | |
| ----- | ----- | |
| **Bold** | <code>`code`</code> in cells | |
| <i>*Italic*</i> | [links](url) | |
| ~~strike~~ | :emoji: | |

What Markdown tables can't do — for any of these, drop into HTML:

Merged cells (colspan/rowspan):

```
html
<table>
  <tr>
    <th colspan="2">Merged Header</th>
    <th>Normal</th>
  </tr>
  <tr>
    <td rowspan="2">Spans 2 rows</td>
    <td>cell</td>
    <td>cell</td>
  </tr>
  <tr>
    <td>cell</td>
    <td>cell</td>
  </tr>
</table>
```

Custom border styles:

```
html
```

```
<table style="border-collapse: collapse; width: 100%;">
  <tr>
    <th style="border: 2px solid black; padding: 8px;">Header 1</th>
    <th style="border: 2px solid black; padding: 8px;">Header 2</th>
  </tr>
  <tr>
    <td style="border: 1px solid #ccc; padding: 8px;">cell</td>
    <td style="border: 1px solid #ccc; padding: 8px;">cell</td>
  </tr>
</table>
```

Note: GitHub strips most CSS for security but allows `border`, `padding`, `width`, `color`, `background-color`, and `text-align` as inline styles. External stylesheets and `<style>` blocks are stripped entirely.

Borderless table (no visible lines):

```
html

<table style="border-collapse: collapse;">
  <tr>
    <td style="border: none; padding: 8px;">cell</td>
    <td style="border: none; padding: 8px;">cell</td>
  </tr>
</table>
```

Rule of thumb: use Markdown syntax for simple data tables; switch to HTML when you need merging, custom borders, or precise control.

Task Lists (GitHub Flavored Markdown)

- [x] Completed item
- [] Incomplete item
- [] Another item

Renders as actual checkboxes on GitHub. Useful in READMEs and issue trackers.

Paragraphs and Line Breaks

A blank line creates a new paragraph (`<p>`). A single newline does nothing — it wraps in the

same paragraph. For a `<br>`, end a line with two spaces or use `
`.

Where Raw HTML Beats Markdown

Markdown has no syntax for these, so you use HTML directly:

Centered content

```
html

<div align="center">
  
  <h1>My Project</h1>
  <p>A short description</p>
</div>
```

Collapsible sections

```
html

<details>
  <summary>Click to expand</summary>

  Content here. You can put Markdown inside – just leave a blank line after the `<su

  ```python
 code works here too
  ```

</details>
```

Extremely common in big READMEs for hiding verbose sections like changelogs, full config examples, or FAQ.

Controlling image size

```
html


```

Markdown's `![]()` syntax has no size control. HTML does.

Making a README Actually Look Good

Badges

Badges are those small colored labels you see at the top of repos — build status, license, version, etc. They come from shields.io and are just images:

markdown

```
![License](https://img.shields.io/badge/license-MIT-blue)
![Python](https://img.shields.io/badge/python-3.11-blue)
![Status](https://img.shields.io/badge/status-active-brightgreen)
```

Custom badge syntax: `(https://img.shields.io/badge/LABEL-MESSAGE-COLOR)`

Colors: `(brightgreen)`, `(green)`, `(yellowgreen)`, `(yellow)`, `(orange)`, `(red)`, `(blue)`, `(lightgrey)`, or hex codes like `(ff69b4)`.

Mermaid Diagrams

GitHub renders [Mermaid](#) diagrams natively — you can draw flowcharts, sequence diagrams, and more in plain text:

```
```mermaid
graph TD
 A[Start] --> B{Decision}
 B -->|Yes| C[Do thing]
 B -->|No| D[Do other thing]
 C --> E[End]
 D --> E
```
```

```
```mermaid
sequenceDiagram
 User->>Server: request
 Server-->>User: response
```
```

Very impressive for architecture diagrams without needing any image files.

GitHub renders `:emoji_name:` syntax:

```
:rocket:   :fire:   :white_check_mark:   :warning:   :bulb:   :gear:
```

Full list at [emojipedia.org](https://www.emojipedia.org) or search "github emoji cheatsheet." Use sparingly in technical docs.

A README Structure That Actually Works

markdown

```
<div align="center">
  <h1>Project Name</h1>
  <p>One sentence description.</p>

  ![License](https://img.shields.io/badge/license-MIT-blue)
  ![Status](https://img.shields.io/badge/status-active-brightgreen)
</div>
```

Overview

2-3 sentences on what this is and why it exists.

Setup

```
\```bash
git clone ...
cd project
./install.sh
\```
```

Usage

Brief example.

Structure

What's in the repo and why.

Escaping

Backslash escapes any Markdown character:


```
\*not italic\*  
\# not a heading  
\`not code\`
```

Markdown Flavors

Not all Markdown is identical. The spec was vague, so variants emerged:

- **CommonMark** — the standardized spec; most renderers follow this
- **GitHub Flavored Markdown (GFM)** — CommonMark + tables, task lists, strikethrough, autolinks, `@mentions`, `#issue` references. This is what your README uses.
- **Pandoc Markdown** — extended for academic writing (footnotes, citations, LaTeX math)
- **MDX** — Markdown with JSX, used in some documentation frameworks

For dotfiles and GitHub, GFM is all you need to know. Everything in this document works in GFM.

The One Thing to Remember

If you can't do it in Markdown, write the HTML. It always works.